TAgged GEometric History Length (TAGE)

Authors: André Seznec, Pierre Michaud

Presentation By: Trevor Keegan & Kevin Grant

Outline

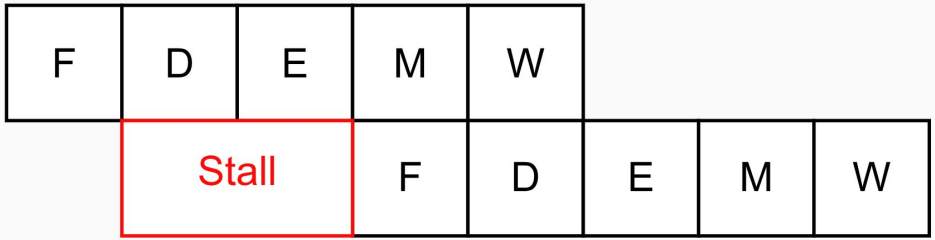
1. Motivation and Challenges
2. Prior approaches: GEHL + PPM Like Predictor
3. TAGE architecture
4. Update Policy and How TAGE Learns
5. Example
6. Results
7. Extensions: ITTAGE & COTTAGE

Motivation & Challenges

- Mispredictions are bad → pipeline flush
- Conditional branches

  ```
  beq r6, r7, TARG
  ```

- Global Branch History (includes outcomes of recent branches)
- Short histories vs. long histories (application specific)

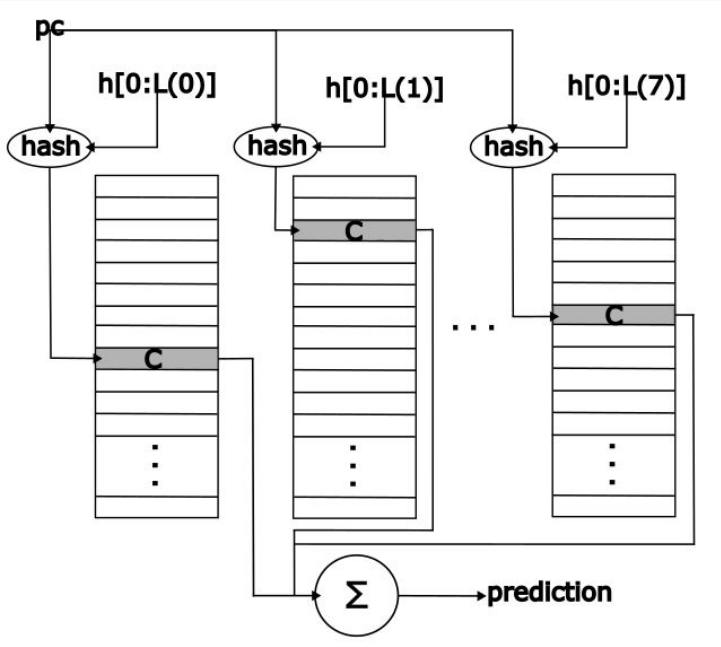


Composition of TAGE

TAGE = GEHL geometric histories + PPM-like partial tagging + improved update policy

Geometric Histories - GEHL and O-GEHL

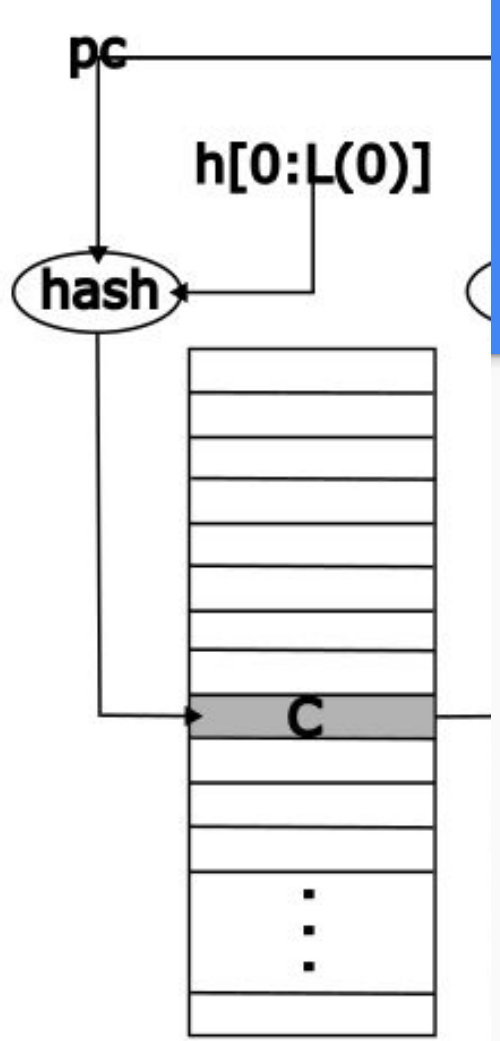
- Some number of tables
  - Entries indexed by hash(PC+history)
  - Each table hashes a different history length
  - History length geometrically increasing
- Entries have a saturation counter
  - Ex: 4-bit saturation counter
  - 0000 = weakest taken
  - 0111 = strongest taken
  - 1000 = strongest not taken
- Counters summed to get prediction



[2] Seznec. “Analysis of the O-geometric history length branch predictor.”

Geometric Histories - GEHL and

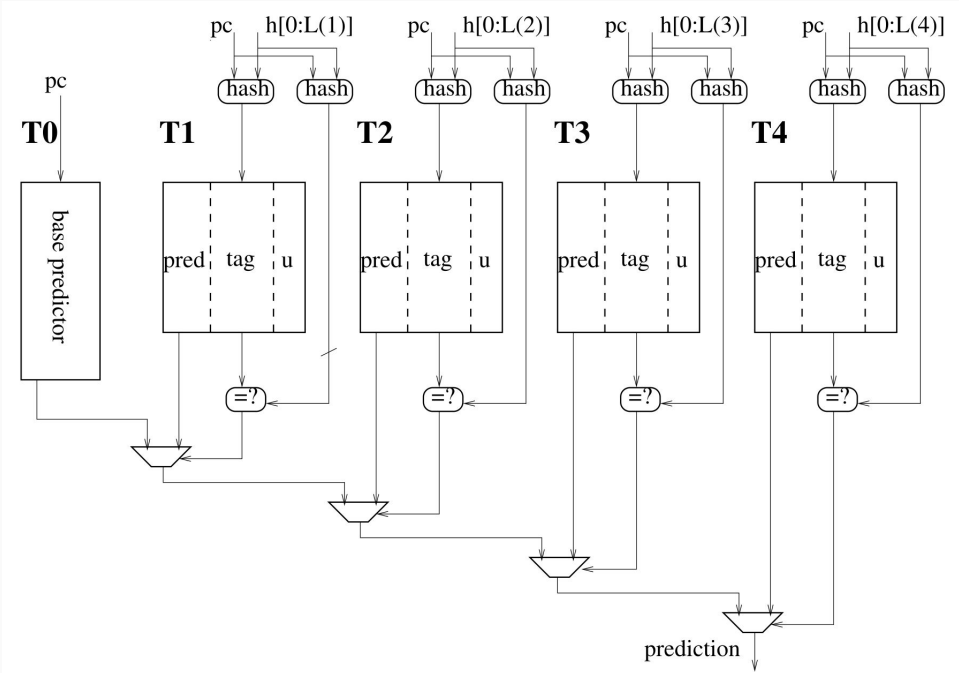
- Some number of tables
  - Entries indexed by hash(PC+history)
  - Each table hashes a different history length
  - History length geometrically increasing
- Entries have a saturation counter
  - Ex: 4-bit saturation counter
  - 0000 = weakest taken
  - 0111 = strongest taken
  - 1000 = strongest not taken
- Counters summed to get prediction



[2] Seznec. “Analysis of the O-geometric history length branch predictor.”

PPM-like Partial Tagging

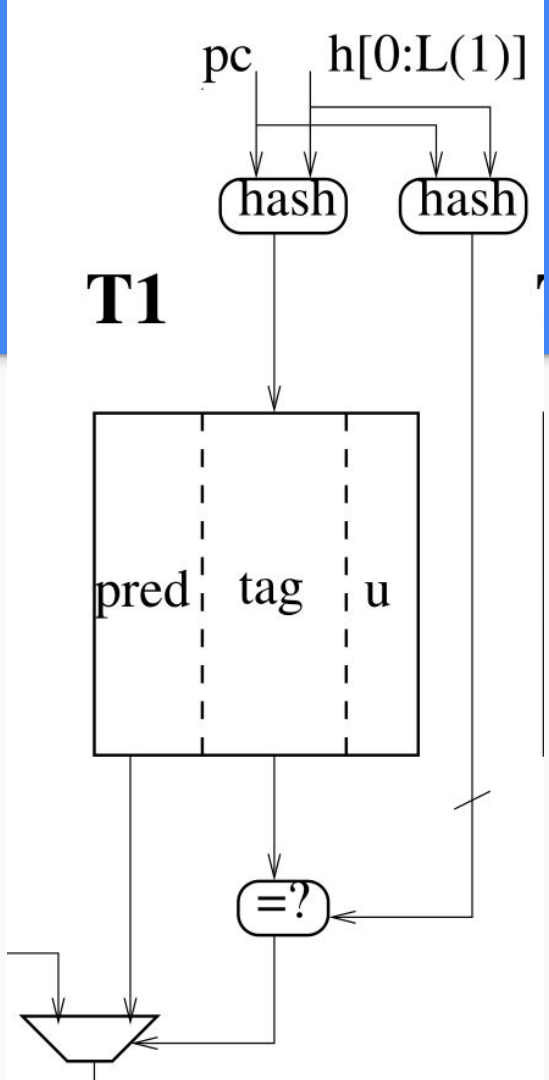
- PPM = Prediction by Partial Matching
- Entries
  - Still indexed by hash(PC+history)
  - Still have saturation counter (pred ex: 3-bit)
  - New tag = hash(PC+history)
  - New usefulness counter (ex: 2-bit)
- Tag width variable, ~10 or 11 bits found to perform well
- New base predictor table
- 5 or 8 tables found to perform well for TAGE



[3] Michaud. “A PPM-like, tag-based branch predictor.”

PPM-like Partial Tagging

- Entries
  - Still indexed by hash(PC+history)
  - Still have saturation counter (pred ex: 3-bit)
  - New tag = hash(PC+history)
  - New usefulness counter (ex: 2-bit)
- Tag width variable, ~10 or 11 bits found to perform well
- New base predictor table
- 5 or 8 tables found to perform well for TAGE



[3] Michaud. “A PPM-like, tag-based branch predictor.”

Improved Update Algorithm - TAGE

- Branch taken → `ctr+=1`
- Branch not taken → `ctr-=1`
- Prediction correct
  - If *pred* different from *altpred* `u+=1`
- Prediction incorrect
  - If *pred* different from *altpred* `u-=1`
  - If *pred* not from table with longest history -> new allocation
- New allocation, search tables with longer histories
  - Replace entry with `u=0`
  - If no entry with `u=0`, decrement u for all entries in tables with longer histories

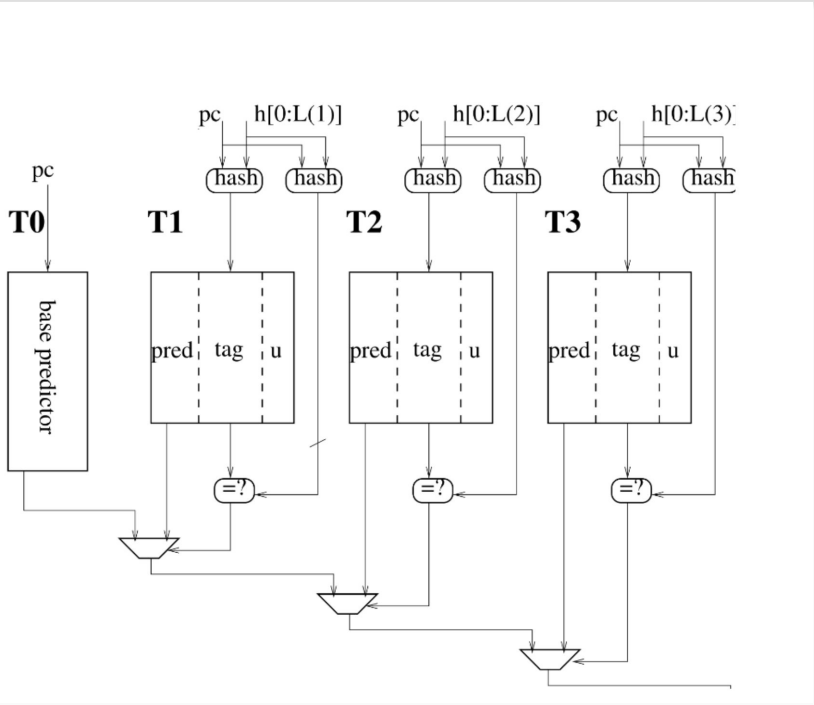
[1] Seznec, Michaud. “A case for (partially) tagged geometric history length branch prediction.”

Example

```
0x140   ld   r5, 0(r1)
0x144   beq  r5, r6, FUNC
0x148   add  r7, r7, r8
…

0x200 FUNC:
          sub r9, r9, r10
```

Current global history: 1 0 1 0 1 1 0 0

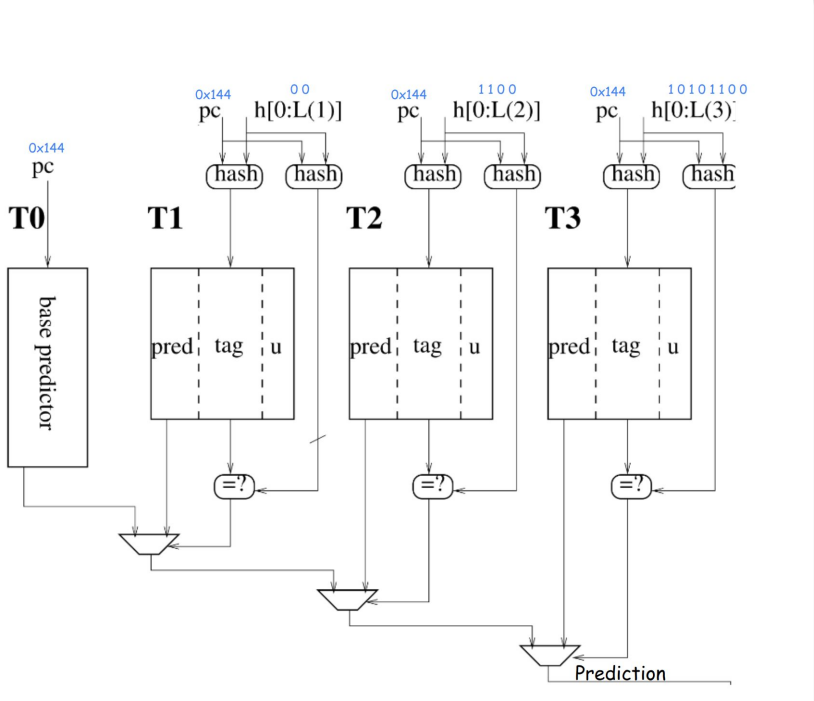


Example

```
        0x140   ld   r5, 0(r1)
        0x144   beq  r5, r6, FUNC
        0x148   add  r7, r7, r8
        …

0x200 FUNC:
              sub r9, r9, r10
```

Current global history: 1 0 1 0 1 1 0 0

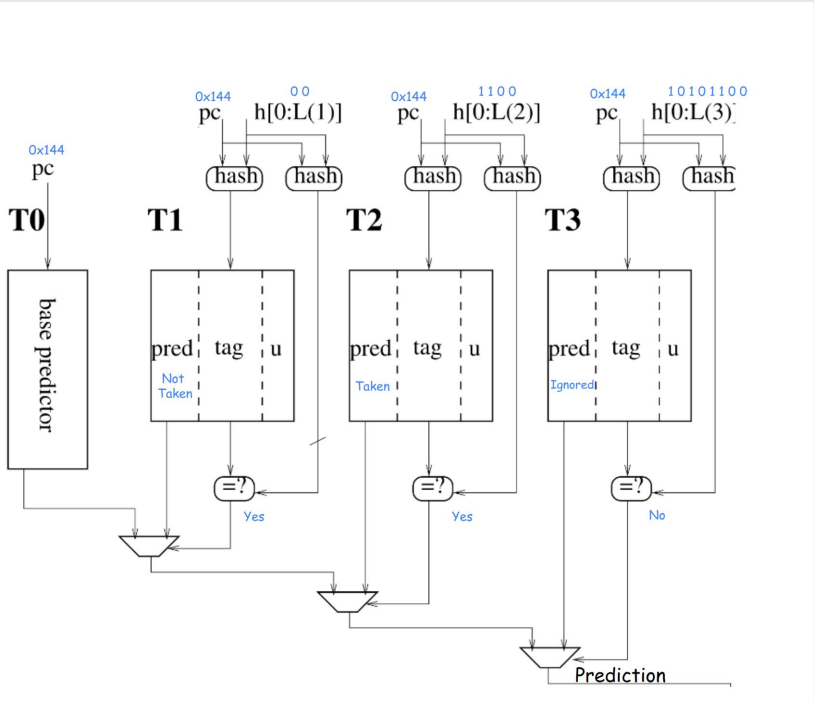


Example

```
        0x140   ld   r5, 0(r1)
        0x144   beq  r5, r6, FUNC
        0x148   add  r7, r7, r8
        …

0x200 FUNC:
              sub r9, r9, r10
```

Current global history: 1 0 1 0 1 1 0 0

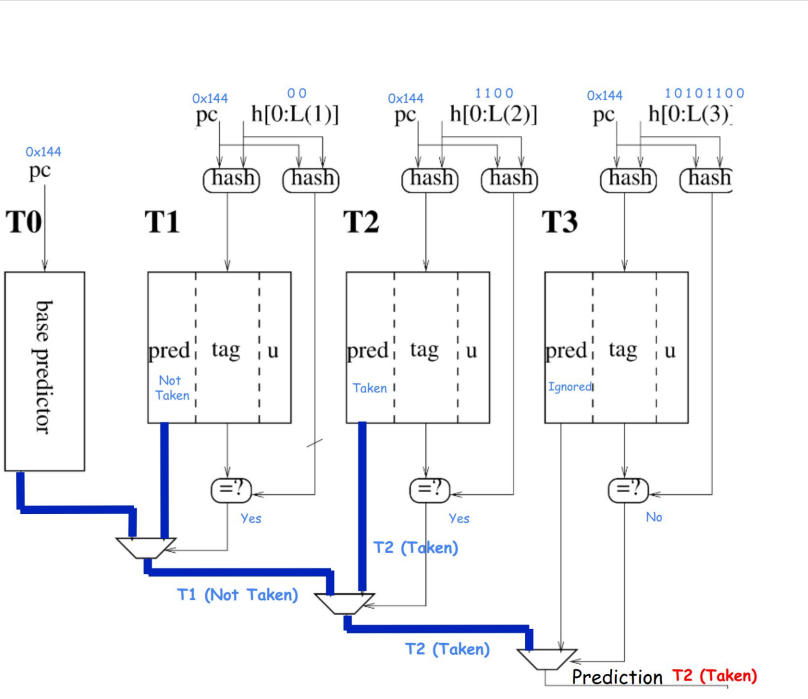


Example

```
    0x140   ld   r5, 0(r1)
    0x144   beq  r5, r6, FUNC
    0x148   add  r7, r7, r8
    …

0x200 FUNC:
          sub r9, r9, r10
```

Current global history: 1 0 1 0 1 1 0 0

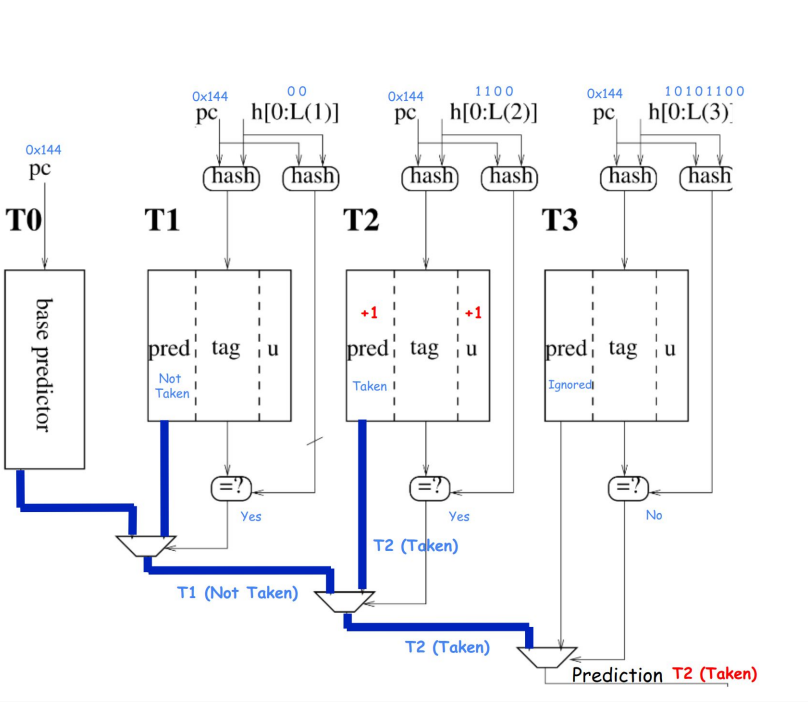


Example

```
    0x140   ld   r5, 0(r1)
    0x144   beq  r5, r6, FUNC
    0x148   add  r7, r7, r8
    …

0x200 FUNC:
          sub r9, r9, r10
```

Current global history: 0 1 0 1 1 0 0 1



Results

1. TAGE reduces mispredictions at the same storage budget

Storage Budget	O-GEHL	TAGE	Reduction in Misprediction
64 Kbits	2.83 MPKI	2.61 MPKI	**7.8%**
1 Mbit	2.27 MPKI	2.05 MPKI	**9.7%**

Results

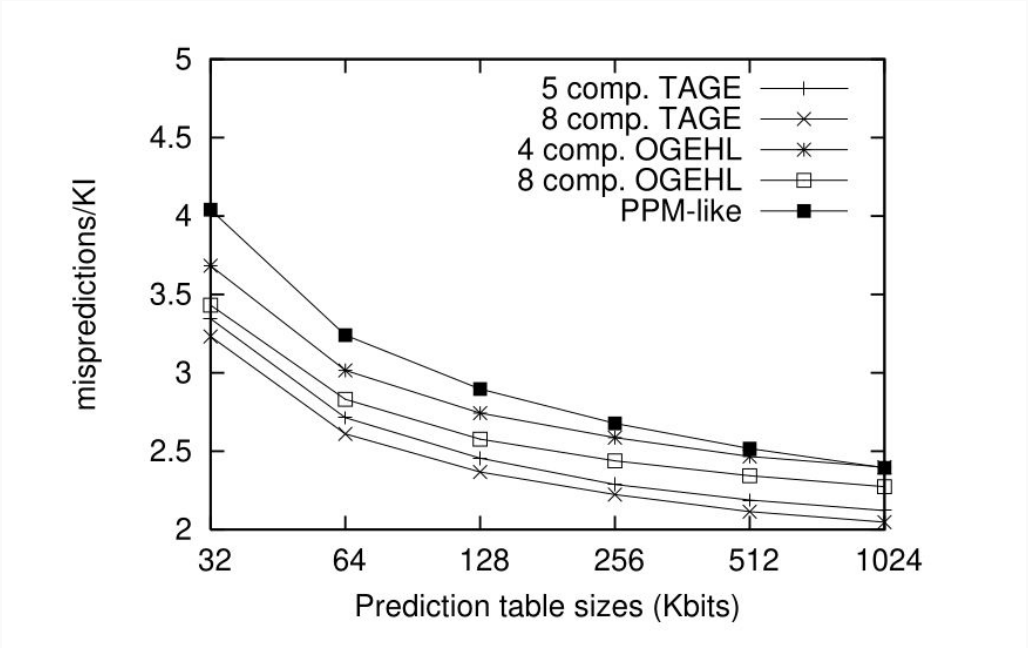
2. TAGE achieves similar accuracy with substantially less storage

Predictor	Storage	MPKI
TAGE	128 Kbits	2.36
O-GEHL	512 Kbits	2.34

4x storage advantage!

Results

3. The update policy is critical



Key Points

- Partial tagging outperforms adder trees at equal memory constraints
- Response time is not faster!
- More than 8 components has marginal benefit
- Championship Branch Prediction (CBP)
  - Storage budget: 64Kbits + 256 bits
  - 4 workload classes: server, multi-media, specint, specfp
  - 30 million instructions ~ 10ms in Windows/Linux machines
  - 2.553MPKI (TAGE) vs 2.820MPKI (best competitors)

Extensions to TAGE

ITTAGE

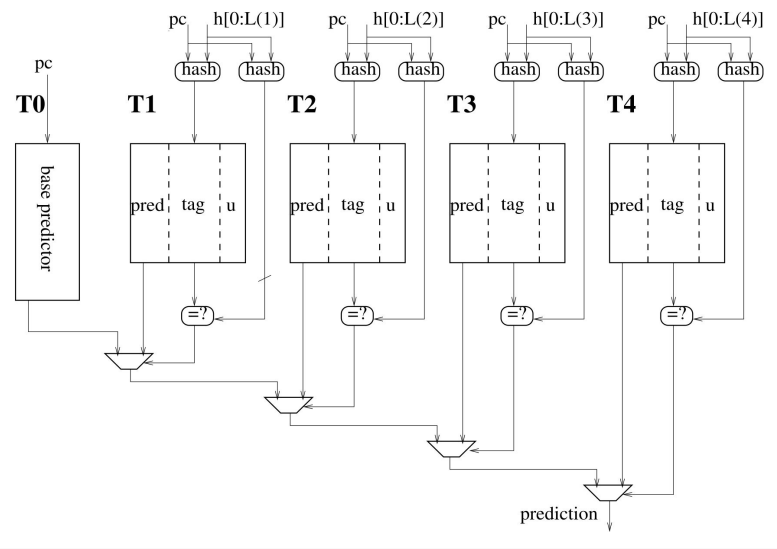
- Indirect branches
  ```
  jal r5
  ```
- Same architecture
- Full target addresses + confidence bit
- No more counters
- Cost-effective?

COTTAGE

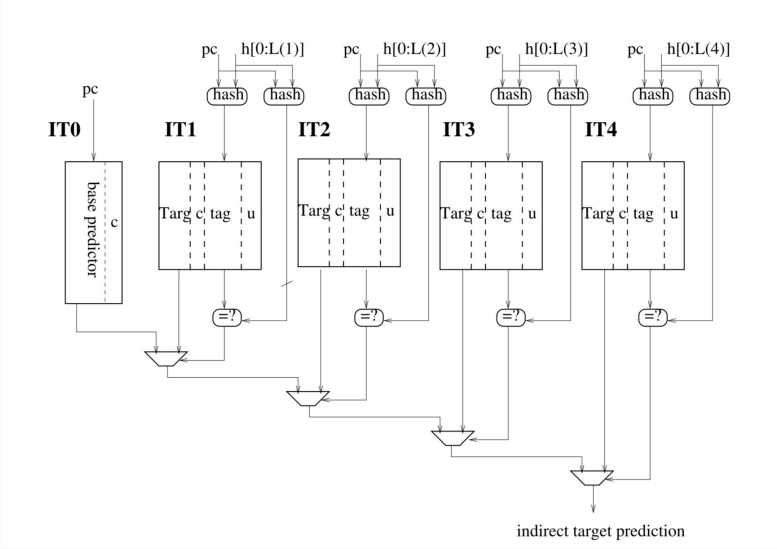
- TAGE + ITTAGE
- Shared tables, index, and tag logic
- More TAGE entries than ITTAGE

ITTAGE Arch

TAGE



ITTAGE



Current State

- TAGE became the foundation for many high-performance production branch predictors, such as Intel’s, AMD’s, and IBM’s modern designs
- Descendants:
  - TAGE-SC (TAGE + Statistical Corrector)
  - TAGE-SC-L (TAGE + Statistical Corrector + Loop Predictor)
  - L-TAGE (Loop-Augmented)

References

[1] André Seznec, Pierre Michaud. “A case for (partially) tagged geometric history length branch prediction.” The Journal of Instruction-Level Parallelism, 2006, 8, pp.23. ⟨hal-03408381⟩

[2] Seznec, Andr´e. “Analysis of the O-geometric history length branch predictor.” 32nd International Symposium on Computer Architecture (ISCA’05), 2005, pp. 394–405, https://doi.org/10.1109/isca.2005.13.

[3] Pierre Michaud. “A PPM-like, tag-based branch predictor.” The Journal of Instruction-Level Parallelism, 2005, 7, pp.10. ⟨hal-03406188⟩